Especificación de Requerimientos de Software (ERS)

Sistema de Gestión y Venta de Gas — "Distribuidora de Gas El Volcán"

Asignatura: DSY1104 — Desarrollo Fullstack II **Evaluación:** Evaluación Parcial N° 3 (integración frontend React + microservicios Spring Boot) **Tecnología principal de esta entrega:** React (SPA) + microservicios Spring Boot + Oracle Cloud **Versión del documento:** 3.0 **Fecha:** Noviembre 2026

Control de versiones

Versión	Fecha	Descripción	Autor
1.0	EP1	ERS inicial del sitio estático (HTML/CSS/JS)	Equipo
2.0	EP2	La solución se reespecifica como SPA en React; se agregan requisitos de testing y arquitectura de componentes	Equipo
2.1	EP2	Actualización de la interfaz (identidad visual, diseño responsive, CSS modular) y del catálogo unificado	Equipo
3.0	**EP3**	**Backend de 4 microservicios Spring Boot + Eureka con JWT, roles ADMIN / VENDEDOR / CLIENTE, API versionada (v1 / v2 HATEOAS), base Oracle Cloud y frontend conectado a la API. Se reemplaza `localStorage` como base de datos; se redefinen los roles; se agregan los RF-38 a RF-47 y el modelo de datos real.**	**Equipo**

---

1. Introducción

1.1 Propósito

Este documento especifica los **requerimientos funcionales y no funcionales** del sistema de gestión y venta de gas de *Distribuidora de Gas El Volcán*. La versión 3.0 **actualiza el ERS de la EP2**, donde la aplicación React guardaba los datos en el navegador. Ahora refleja la solución **full stack**:

- un frontend React que consume **microservicios Spring Boot** por API REST/JSON;
- autenticación con **JWT**;
- control de acceso por **roles** aplicado en el servidor;
- persistencia en **Oracle Autonomous Database**.

El ERS sirve como:

- Base para el diseño y la implementación del frontend y del backend.
- Referencia para las pruebas unitarias, de integración y manuales.
- Acuerdo entre los interesados (clientes, vendedores y administración) y el equipo de desarrollo.

1.2 Alcance de esta entrega

Dentro del alcance (EP3):

- **4 microservicios Spring Boot + Eureka Server:**
  - `ms-auth`: usuarios, roles y JWT.

  - `ms-catalogoProductos`: productos y precios.
  - `ms-inventario`: stock.
  - `ms-ventas`: ventas y pagos.

  Se comunican con **OpenFeign** y descubrimiento en **Eureka**.
- **Seguridad:** JWT, contraseñas con **BCrypt**, autorización por rol (`@PreAuthorize` y reglas en `SecurityConfig`), **CORS** configurable y secretos fuera del repositorio.
- **API versionada:** `/api/v1` (REST con DTOs) y `/api/v2` (HATEOAS), documentadas con **Swagger / OpenAPI**.
- **CRUD** de productos, inventario y usuarios.
- **Frontend React conectado a la API:**
  - catálogo, carrito, checkout real y "Mis compras";
  - registro y login contra ms-auth;
  - rutas protegidas por rol;
  - **panel del Vendedor de solo lectura**;
  - panel de administración con datos reales (inventario, órdenes, usuarios, clientes y reportes con exportación CSV).
- **Pruebas automatizadas:** Vitest en el frontend y JUnit, Mockito y `@WebMvcTest` en el backend, con reporte de cobertura (V8 y JaCoCo).
- Persistencia en **Oracle Autonomous Database (Oracle Cloud, con Wallet)**.

Fuera del alcance de esta entrega:

- `ms-produccion`: microservicio heredado del template MiniMarket, no adaptado.
- **Seguimiento de pedidos en tiempo real** con mapas y roles de despacho (Operadora y Repartidor).
- **Pasarela de pago real:** el pago se registra con la confirmación de la venta.
- Despliegue de los microservicios en **AWS con Docker**. El frontend de la V2 ya se desplegó en EC2 (ver `DEPLOY.md`).

1.3 Definiciones y acrónimos

Término	Significado
SPA	*Single Page Application*; aplicación de una sola página que se actualiza dinámicamente.
Microservicio	Aplicación independiente con su propio puerto y su propia responsabilidad, que expone una API REST.
Eureka	Servidor de descubrimiento: cada microservicio se registra con un nombre (ej. `MS-VENTAS`).
OpenFeign	Cliente HTTP declarativo para que un microservicio llame a otro por su nombre en Eureka.
JWT	*JSON Web Token*; token firmado que identifica al usuario y su rol en cada petición.
DTO	*Data Transfer Object*; objeto con los datos que entran o salen de la API (sin exponer la entidad).
HATEOAS	Respuestas con enlaces (`_links`) a las acciones relacionadas; se usa en la API v2.
CORS	Mecanismo que permite al navegador llamar a una API de otro origen (puerto).
RBAC	*Role-Based Access Control*; control de acceso basado en roles.
CRUD	Crear, Leer, Actualizar y Eliminar.

Término	Significado
RF / RNF	Requerimiento Funcional / No Funcional.
SKU	Código único del producto; enlaza el catálogo con el inventario.

1.4 Referencias

- Documento de contexto del proyecto *DSY1104 — Forma C — Distribuidora de Gas El Volcán*.
- Instrucciones y rúbrica de la *Evaluación Parcial N° 3* (DSY1104).
- Anexos de la EP3 (`docs/parcial3/`): Documentación de APIs e Integración (Anexo 2), Documentación de APIs en Excel (Anexo 3) y Manual de usuario (Anexo 4).
- `DOCUMENTO_COBERTURA_TESTING.md` , `DOCUMENTACION.md` y `microservicios/README.md` (guía de la API).

---

2. Descripción general

2.1 Contexto del negocio

Distribuidora de Gas El Volcán es una empresa familiar ubicada en **Chillán, Región de Ñuble**, fundada en 1998. Distribuye cilindros de gas licuado (5, 11, 15 y 45 kg) y accesorios a clientes **residenciales y comerciales**. Cuenta con 2 camiones, 3 repartidores, 1 operadora de llamadas y 1 administradora, y recibe entre **80 y 120 pedidos diarios**.

El proceso actual es **manual** (teléfono y cuadernos). Esto genera:

- clientes sin información del estado de su pedido;
- pedidos duplicados o perdidos;
- falta de historial;
- descontrol del stock y diferencias de caja.

El sistema **digitaliza** el catálogo, los pedidos, el stock y la administración.

2.2 Objetivos del sistema

1. Permitir que los clientes **vean el catálogo** y **compren** de forma autónoma, con historial de sus compras.
2. Dar a la administración una vista **centralizada y en tiempo real** de productos, stock, órdenes, usuarios y reportes.
3. **Descontar el stock automáticamente** al confirmar cada venta, evitando vender sin existencias.
4. Aplicar **control de acceso por rol en el servidor** (RBAC con JWT).
5. Ofrecer una experiencia **responsive**, simple y consistente.

2.3 Actores y roles (RBAC)

Rol	Descripción	Permisos (aplicados en el backend y reflejados en el frontend)
Administrador (`ADMIN`)	Gestiona el sistema.	**Acceso total:** CRUD de productos e inventario, CRUD de usuarios (elige el rol), todas las órdenes, dashboard, clientes, reportes y exportación CSV.
Vendedor (`VENDEDOR`)	Atiende y consulta pedidos.	**Solo lectura:** lista y detalle de productos (con stock) y lista y detalle de órdenes. **Nada más le aparece**: la tienda lo redirige a su panel y el backend responde 403 a cualquier escritura.

Rol	Descripción	Permisos (aplicados en el backend y reflejados en el frontend)
Cliente (`CLIENTE`)	Compra cilindros y accesorios.	**Solo la tienda:** catálogo, carrito, compra, "Mis compras" y pago de **sus** órdenes pendientes.

> El registro público siempre crea usuarios **CLIENTE**. Los Vendedores y Administradores los crea un ADMIN. Los roles *Operadora* y *Repartidor* de la versión 2.1 se reemplazan por el **Vendedor**; el despacho queda para una etapa posterior.

2.4 Supuestos y dependencias

- Los microservicios necesitan **Java 21**, la base **Oracle Autonomous Database** (con Wallet) y el archivo de secretos `~/.elvolcan/secrets.properties` .
- Eureka debe estar activo antes que los demás servicios (lo resuelve `levantar-servicios.ps1`).
- Todos los servicios comparten la **misma clave JWT**, así que el token emitido por ms-auth es válido en el resto.
- Si un microservicio no responde, el frontend lo **informa** al usuario y deshabilita la compra (no falla en silencio).

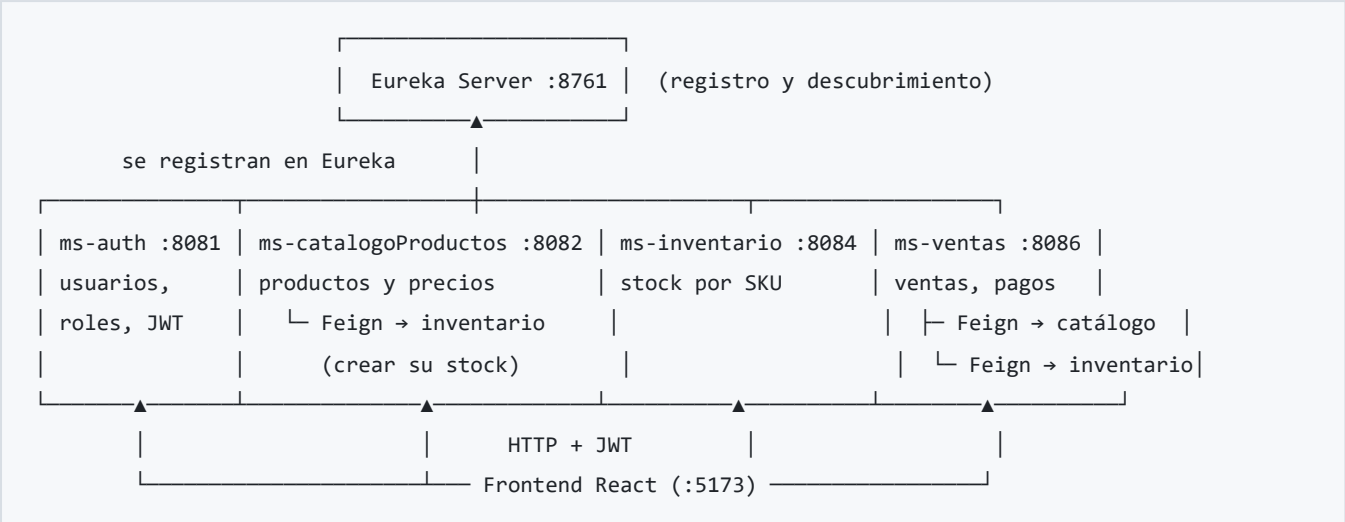
---

3. Arquitectura de la solución

3.1 Stack tecnológico

Capa	Tecnología	Rol
Frontend	**React 19 (SPA)** + Vite 8	Interfaz completa.
Diseño	**Bootstrap 5 + React Bootstrap + CSS modular**	Responsive (≥360, ≥768, ≥1280 px).
Ruteo	**React Router 7**	Navegación, rutas anidadas y **rutas protegidas por rol**.
Consumo de API	`fetch` + JWT en `Authorization: Bearer`	Llamadas REST a los microservicios.
Backend	**Spring Boot 4.0.6 (Java 21)**	4 microservicios REST/JSON.
Descubrimiento / comunicación	**Spring Cloud 2025.1.1**: Eureka + OpenFeign	Registro de servicios y llamadas entre ellos.
Seguridad	**Spring Security + JWT (jjwt) + BCrypt**	Autenticación y autorización por rol.
Persistencia	**Spring Data JPA + Oracle Autonomous Database**	Un esquema por microservicio.
Documentación API	**springdoc-openapi (Swagger UI)**	Grupos `v1` y `v2` en cada servicio.
Pruebas	**Vitest** (front) · **JUnit 5 + Mockito + MockMvc** (back)	Cobertura con V8 y JaCoCo.
Mapas	Google Maps embebido	Ubicación de sucursales (Tiendas).

3.2 Arquitectura de microservicios



- Cada microservicio tiene su **propia configuración y sus tablas** en Oracle Cloud. El frontend **no accede a la base de datos**: solo a la API.
- **Flujo de compra:**
  1. `POST /api/v1/ventas`: ms-ventas consulta el **precio** en ms-catalogoProductos y el **stock** en ms-inventario. Si no alcanza, responde 409.
  2. `PUT /api/v1/ventas/{id}/confirmar`: marca la venta PAGADA y **descuenta el stock** en ms-inventario, con un token interno de servicio.

3.3 Arquitectura del frontend (componentes)

```
main.jsx
└── App (carrito + enrutador)
    ├── LayoutPublico → BotonTienda + Navbar (según rol) + <Outlet/> + Footer
    │   ├── Inicio, Productos → Producto, DetalleProducto, Carrito
    │   ├── RutaProtegida[Cliente, Admin] → Ventas (checkout), MisCompras
    │   ├── Nosotros, Consejos, Tiendas, Contacto
    │   └── Login, Registro
    ├── RutaProtegida[Admin] → AdminLayout → Dashboard, Inventario, Ordenes,
    │                                        ListaUsuarios, NuevoUsuario, Clientes, Reportes
    └── RutaProtegida[Vendedor, Admin] → VendedorLayout → ProductosVendedor, DetalleProductoVendedor,
                                                          OrdenesVendedor, DetalleOrdenVendedor
```

Utilidades:

- `utils/api.js`: cliente REST con `ApiError` (conserva el código HTTP).
- `utils/sesion.js`: sesión, roles y expiración del JWT.
- `utils/useCatalogo.js`: hook del catálogo.
- `utils/productos.js`, `carrito.js`, `inventario.js`, `ventas.js`, `reportes.js` y `validaciones.js`: lógica pura y probada.

3.4 Seguridad

- **Autenticación:**

    - `POST /api/v1/auth/login` valida con **BCrypt** y emite un **JWT** con el rol, válido por 10 horas.
    - El frontend guarda el token en la sesión y lo envía en cada petición.
    - Si el token expiró, se pide volver a iniciar sesión.
- **Autorización en el backend:**
    - `@PreAuthorize` y reglas por método HTTP en cada `SecurityConfig`.
    - El servicio de ventas verifica además que el cliente sea **dueño** de la venta.
    - Respuestas: **401** sin token o con credenciales malas; **403** sin permiso.
- **Autorización en el frontend:** `RutaProtegida` y el Navbar muestran solo las vistas del rol. Un Vendedor en la tienda es redirigido a su panel.
- **Registro:** el body **no** decide el rol: siempre se crea un CLIENTE.
- **Secretos:** la contraseña de la base, la ruta del Wallet y la clave JWT se leen de un archivo externo. El Wallet **no** se versiona.
- **CORS:** solo para `/api/**` y los orígenes configurados (el frontend local).

3.5 Versionado de la API

- **`/api/v1/...`:** REST con DTOs. La usa el frontend.
- **`/api/v2/...`:** las mismas operaciones con **HATEOAS** (`_links`, `_embedded`).
- En cada Swagger se elige el grupo **"API v1 (REST)"** o **"API v2 (HATEOAS)"**. El detalle de cada endpoint está en los Anexos 2 y 3.

3.6 Identidad visual y diseño de la interfaz

Se mantiene la identidad de la V2:

- navegación *sticky* (`navbar-gas`) con la pestaña activa resaltada;
- portada con cilindros y 4 beneficios con íconos;
- tarjetas de producto y CSS modular (`navbar.css`, `productos.css`, `carrito.css`).

Los paneles del Administrador y del Vendedor usan un layout con menú lateral colapsable.

3.7 Catálogo único (público ↔ administración ↔ inventario)

El catálogo vive en **ms-catalogoProductos**:

- Lo que el Administrador crea o edita se refleja de inmediato en la tienda, el detalle, el carrito, el checkout y los paneles.
- Cada producto se enlaza con su stock en **ms-inventario** por el **SKU**, que por eso no se puede modificar.
- La semilla inicial contiene los **14 productos** de la empresa (CL001…AC003).

---

4. Requisitos funcionales (RF)

Prioridad: A = Alta · M = Media · B = Baja.

Estado:

- ✅ Implementado y conectado a la API (EP3).
- ✅[1] Implementado en el frontend, sin microservicio propio.
- ⬜ Planificado.

4.1 Módulo público y catálogo

Cód.	Requerimiento	Prioridad	Estado
RF-01	El sistema debe mostrar una página de **inicio** con bienvenida, imagen de cilindros, beneficios del servicio y acceso al catálogo.	A	✅
RF-02	El sistema debe mostrar el **catálogo público** obtenido de ms-catalogoProductos (`GET /api/v1/productos` , sin autenticación), en tarjetas con imagen, nombre, descripción y precio.	A	✅
RF-03	El usuario debe poder **filtrar** los productos por **categoría** (Cilindros de Gas, Reguladores, Mangueras y Conexiones, Accesorios) y ver "Todos".	A	✅
RF-04	El usuario debe poder abrir el **detalle** de un producto (código, categoría, precios residencial y comercial) desde la API.	A	✅
RF-05	El sistema debe indicar claramente cuando un producto **no existe** (404).	M	✅
RF-06	El sistema debe mostrar los precios en **pesos chilenos** (separador de miles).	M	✅
RF-37	Todo producto **creado o editado** por el administrador debe reflejarse de inmediato en el catálogo público, el detalle, el carrito, el checkout y los indicadores del panel.	A	✅

4.2 Módulo carrito

Cód.	Requerimiento	Prioridad	Estado
RF-07	El usuario debe poder **agregar** un producto al carrito desde el catálogo y desde el detalle.	A	✅
RF-08	El usuario debe poder **aumentar** y **disminuir** la cantidad de cada ítem.	A	✅
RF-09	Al disminuir a **cero**, el ítem debe **eliminarse** del carrito.	M	✅
RF-10	El sistema debe **calcular el total** con tarifa **residencial** o **comercial** según el tipo de cliente. El backend recalcula el precio al crear la venta.	A	✅
RF-11	La barra de navegación debe mostrar el **contador de ítems** del carrito.	M	✅
RF-12	El carrito debe **persistir** entre recargas (en el navegador, con los ids de la API).	M	✅
RF-13	El sistema debe indicar cuando el carrito está **vacío**.	B	✅

4.3 Módulo de cuenta (registro / sesión)

Cód.	Requerimiento	Prioridad	Estado
RF-14	El usuario debe poder **registrarse** como cliente (Residencial o Comercial) en ms-auth (`POST /api/v1/auth/register`).	A	✅
RF-15	El sistema debe **validar** RUN (7–9 dígitos), teléfono, formato y dominio de correo, contraseña (**6–10**) y términos. El backend vuelve a validar (`@Valid`).	A	✅

Cód.	Requerimiento	Prioridad	Estado
RF-16	El sistema debe **impedir correos o RUN duplicados** en el registro (409).	A	✅
RF-17	El usuario debe poder **iniciar sesión** contra ms-auth y ser dirigido según su rol: Administrador a `/admin`, Vendedor a `/vendedor`, Cliente a la tienda.	A	✅
RF-18	El sistema debe **cerrar sesión** eliminando el token y los datos de sesión.	A	✅
RF-19	El sistema debe usar **selectores dependientes** Región → Comuna.	M	✅

4.4 Módulo de pedido / checkout

Cód.	Requerimiento	Prioridad	Estado
RF-20	Para comprar, el usuario debe tener **sesión de Cliente o Administrador**; si no, se le envía al login y luego vuelve al checkout.	A	✅
RF-21	El usuario debe poder **revisar** su pedido (ítems, cantidades y subtotales) antes de confirmar.	A	✅
RF-22	El usuario debe ingresar **dirección de despacho** y **comuna**, y elegir **método de pago** (Efectivo o Tarjeta). Los tres datos se guardan en la venta.	A	✅
RF-23	El sistema debe **registrar la venta** en ms-ventas con número, fecha, cliente, despacho, ítems y total, validando el **stock** en ms-inventario (409 si no alcanza).	A	✅
RF-24	Al **confirmar el pago**, la venta pasa a PAGADA, se **descuenta el stock** y se muestra la confirmación con el número de orden. Si la confirmación falla, la venta queda pendiente y se puede **reintentar sin duplicarla**.	A	✅
RF-25	El sistema debe mostrar al cliente el seguimiento del despacho en tiempo real con mapas.	M	⬜

4.5 Módulo de contacto e información

Cód.	Requerimiento	Prioridad	Estado
RF-26	El usuario debe poder enviar un **mensaje de contacto** con validación.	M	✅[1]
RF-27	El sistema debe **registrar las solicitudes** de contacto. Hoy se guardan en el navegador; no hay microservicio de contacto.	M	✅[1]
RF-28	El sistema debe ofrecer páginas informativas: **Nosotros**, **Consejos** y **Tiendas** (con mapa).	B	✅

4.6 Módulo de administración

Cód.	Requerimiento	Prioridad	Estado
RF-29	El panel debe tener un **dashboard** con indicadores reales (ingresos de ventas pagadas, órdenes, pendientes y productos en stock crítico) y las últimas órdenes.	A	✅
RF-30	El administrador debe poder **listar el inventario** (catálogo + stock por SKU), resaltando el **stock crítico**.	A	✅

Cód.	Requerimiento	Prioridad	Estado
RF-31	El administrador debe poder **crear** productos con validación. El producto se crea en el catálogo, se crea su registro de inventario y se carga su stock inicial.	A	✅
RF-32	El administrador debe poder **editar** productos (`PUT /api/v1/productos/{id}`), excepto el SKU, y **ajustar el stock**.	A	✅
RF-33	El administrador debe poder **eliminar** productos (borrado lógico: dejan de mostrarse en la tienda).	M	✅
RF-34	El administrador debe poder **listar y filtrar usuarios** por rol (Administradores, Clientes y Vendedores).	A	✅
RF-35	El administrador debe poder **crear usuarios** con rol (Admin, Vendedor o Cliente), región y comuna, con validaciones y control de duplicados (409).	A	✅
RF-36	El SKU de un producto no debe poder modificarse, porque enlaza el catálogo con el inventario.	M	✅

4.7 Roles, órdenes y API (nuevos en EP3)

Cód.	Requerimiento	Prioridad	Estado
RF-38	El sistema debe autenticar con **JWT** emitido por ms-auth y validarlo en todos los microservicios.	A	✅
RF-39	El **Vendedor** debe ver **solo** la lista y el detalle de productos y de órdenes (panel `/vendedor`). No le aparece ninguna otra vista y el backend le niega cualquier escritura (403).	A	✅
RF-40	El **Cliente** debe acceder solo a la tienda y a **sus** compras. El backend le niega ver o pagar ventas ajenas (403).	A	✅
RF-41	El **Administrador** debe tener acceso total a todas las vistas y operaciones.	A	✅
RF-42	El cliente debe poder consultar su historial en **"Mis compras"** y pagar las órdenes pendientes.	A	✅
RF-43	El administrador debe poder ver **todas las órdenes** con indicadores y su distribución por comuna; el Vendedor y el Administrador, el **detalle** de cada orden.	A	✅
RF-44	El administrador debe poder ver **Clientes** (compras, mejores clientes, comunas) y **Reportes** (ingresos, ticket promedio, ventas mensuales, por categoría, más vendidos), calculados con datos reales y **exportables a CSV**.	M	✅
RF-45	Todas las rutas de la API deben estar **versionadas** (`/api/v1` , `/api/v2` con HATEOAS) y documentadas en **Swagger**.	A	✅
RF-46	Si un microservicio no responde, el frontend debe **avisarlo** y deshabilitar la compra (sin fallar en silencio).	M	✅
RF-47	La sesión debe expirar con el token: con un JWT vencido, el sistema debe pedir iniciar sesión de nuevo.	M	✅

---

5. Requisitos no funcionales (RNF)

Cód.	Categoría	Requerimiento
RNF-01	**Responsive**	La interfaz debe verse bien en móvil (≥360 px), tableta (≥768 px) y escritorio (≥1280 px). Menús en "hamburguesa" y grillas adaptables.
RNF-02	**Usabilidad**	Formularios de pocos pasos; errores en español, claros, incluido el motivo que envía el backend (ej. "Stock insuficiente para: …").
RNF-03	**Compatibilidad**	Navegadores modernos (Chrome, Edge, Firefox) y teléfonos Android de gama media-baja.
RNF-04	**Rendimiento**	Carga rápida; bundle de producción optimizado; los paneles cargan sus datos en paralelo.
RNF-05	**Seguridad**	Toda operación que no sea pública (catálogo, login y registro) exige JWT. Contraseñas con BCrypt. Roles aplicados en el servidor. Secretos fuera del repositorio. CORS restringido.
RNF-06	**Confiabilidad**	Errores controlados con códigos HTTP coherentes (400 / 401 / 403 / 404 / 409). Las fallas parciales de un servicio no bloquean todo el panel.
RNF-07	**Mantenibilidad**	Capas controller / service / repository con DTOs; componentes y utilidades reutilizables; lint sin errores.
RNF-08	**Testabilidad**	La lógica de negocio y las reglas de seguridad deben tener **pruebas automatizadas** con reporte de cobertura (frontend y backend).
RNF-09	**Accesibilidad**	Etiquetas asociadas a los campos, `role` y `aria-label` en navegación, botones y alertas.
RNF-10	**Escalabilidad**	Microservicios independientes con descubrimiento en Eureka; se pueden agregar servicios sin cambiar los existentes.
RNF-11	**Disponibilidad**	Despliegue planificado en **AWS** con **Docker** (el frontend de la V2 ya se desplegó en EC2).
RNF-12	**Privacidad**	Los datos de clientes y pedidos solo los ve el personal autorizado y cada cliente ve solo los suyos. La API nunca devuelve contraseñas.
RNF-13	**Interoperabilidad**	API REST/JSON documentada con OpenAPI (Swagger), versionada en v1 y v2.

---

6. Modelo de datos

Cada microservicio administra **sus propias tablas** en Oracle Cloud. Las relaciones **entre** servicios no son claves foráneas de la base de datos: se resuelven por la API, mediante el `username`, el `id` de producto o el SKU.

Microservicio	Tabla (entidad JPA)	Atributos	Relaciones
ms-auth	**roles** (`Rol`)	`id` (PK), `nombre_rol` (único: ADMIN, VENDEDOR, CLIENTE)	1 rol → N usuarios
ms-auth	**usuarios** (`Usuario`)	`id` (PK), `nombre`, `rut` (único), `username` (único, correo), `password` (hash BCrypt), `rol_id` (FK → roles), `direccion`, `region`, `comuna`, `tipo_cliente`, `telefono`	N usuarios → 1 rol

Microservicio	Tabla (entidad JPA)	Atributos	Relaciones
ms-catalogoProductos	**productos** (`Producto`)	`id` (PK), `sku` (único), `nombre`, `descripcion`, `precio_residencial`, `precio_comercial`, `precio`, `categoria`, `imagen`, `activo` (borrado lógico)	1 producto ↔ 1 inventario (por `sku`)
ms-inventario	**inventarios** (`Inventario`)	`id` (PK), `sku` (único), `stock_actual`, `nivel_critico`, `nivel_maximo`, `ultima_actualizacion`	por `sku` con productos
ms-ventas	**ventas** (`Venta`)	`id` (PK), `usuario` (username del cliente), `total`, `estado` (PENDIENTE_PAGO, PAGADA, CANCELADA), `fecha_venta`, `direccion`, `comuna`, `metodo_pago`	1 venta → N detalles
ms-ventas	**detalles_venta** (`DetalleVenta`)	`id` (PK), `venta_id` (FK → ventas), `producto_id` (id del catálogo), `cantidad`, `precio_unitario`, `subtotal`	N detalles → 1 venta

```
ms-auth:      roles 1──N usuarios
ms-catalogo:  productos (sku) ·····  ms-inventario: inventarios (sku)
ms-ventas:    ventas 1──N detalles_venta ··  producto_id → productos.id (por API)
              ventas.usuario ·····  usuarios.username (por API, JWT)
```

> Las solicitudes de contacto se guardan en el navegador (RF-27): no hay tabla de contacto.

---

7. Interfaces externas

7.1 API REST (v1; la v2 expone las mismas operaciones con HATEOAS)

Servicio	Método	Ruta	Descripción	Acceso
ms-auth	`POST`	`/api/v1/auth/login`	Inicia sesión y devuelve el JWT y los datos del usuario	Público
ms-auth	`POST`	`/api/v1/auth/register`	Registra un cliente	Público
ms-auth	`GET` / `POST`	`/api/v1/usuarios`	Lista y crea usuarios con rol	ADMIN
ms-auth	`GET` / `PUT` / `DELETE`	`/api/v1/usuarios/{id}`	Detalle, edición y eliminación	ADMIN
ms-catalogoProductos	`GET`	`/api/v1/productos`, `/id/{id}`, `/{sku}`	Catálogo y detalle	Público
ms-catalogoProductos	`POST` · `PUT` · `DELETE`	`/api/v1/productos` · `/{id}` · `/{sku}`	Crear, editar y desactivar	ADMIN
ms-inventario	`GET`	`/api/v1/inventario`, `/{sku}`, `/critico`	Stock	Autenticado

Servicio	Método	Ruta	Descripción	Acceso
ms-inventario	`POST` · `PUT`	`/crear` · `/aumentar` · `/disminuir`	Gestión de stock	ADMIN
ms-ventas	`POST`	`/api/v1/ventas`	Crea una venta	CLIENTE, ADMIN
ms-ventas	`PUT`	`/api/v1/ventas/{id}/confirmar`	Confirma el pago (solo el dueño o ADMIN)	CLIENTE, ADMIN
ms-ventas	`GET`	`/api/v1/ventas/mis-compras`	Historial propio	CLIENTE, ADMIN
ms-ventas	`GET`	`/api/v1/ventas` , `/{id}`	Todas las órdenes y su detalle	ADMIN, VENDEDOR (el cliente solo el detalle de las suyas)

El detalle completo (parámetros, body, respuestas y códigos de error) está en los **Anexos 2 y 3** y en el Swagger de cada servicio.

7.2 Servicios externos

- **Oracle Autonomous Database** (Oracle Cloud), con conexión por Wallet.
- **Mapas:** Google Maps embebido en *Tiendas*. El seguimiento de entregas con mapas está planificado (RF-25).

---

8. Restricciones

Tipo	Descripción
Conectividad	Zonas con señal intermitente: la interfaz debe tolerar conexiones lentas e informar si un servicio no responde.
Dispositivos	Teléfonos Android de gama media-baja.
Nivel técnico	Usuarios sin experiencia en software de gestión: mínima cantidad de pasos.
Base de datos	Oracle Autonomous Database (Oracle Cloud, plan gratuito: pool de conexiones limitado a 3 por servicio).
Seguridad	Los secretos (contraseña de la BD, clave JWT, Wallet) no se versionan.
Stack	React (SPA) · Spring Boot 4 / Java 21 · Spring Cloud (Eureka, OpenFeign) · API REST/JSON · Oracle Cloud.

---

9. Matriz de trazabilidad (RF ↔ implementación ↔ pruebas)

RF	Implementación	Prueba asociada
RF-01, RNF-01	`pages/Inicio.jsx` , `components/Navbar.jsx`	`Navbar.test.jsx`

RF	Implementación	Prueba asociada
RF-02, RF-03, RF-06, RF-46	`pages/Productos.jsx`, `utils/useCatalogo.js`, `utils/productos.js` · ms-catalogoProductos `ProductoController`	`useCatalogo.test.js`, `productos.test.js`, `Producto.test.jsx` · `ProductoSeguridadTest` (catálogo público)
RF-04, RF-05	`pages/DetalleProducto.jsx`	`DetalleProducto.test.jsx`
RF-07 – RF-13	`utils/carrito.js`, `pages/Carrito.jsx`, `App.jsx`	`carrito.test.js`, `Carrito.test.jsx`, `Navbar.test.jsx`
RF-14 – RF-16, RF-19	`pages/Registro.jsx`, `utils/validaciones.js` · ms-auth `AuthService.registrar`	`Registro.test.jsx`, `validaciones.test.js` · `AuthServiceTest`, `AuthControllerTest` (409, 400)
RF-17, RF-18, RF-38, RF-47	`pages/Login.jsx`, `utils/sesion.js`, `components/RutaProtegida.jsx` · ms-auth `AuthController.login`, `JwtUtil`	`Login.test.jsx`, `sesion.test.js`, `RutaProtegida.test.jsx` · `AuthServiceTest`
RF-20 – RF-24	`pages/Ventas.jsx` · ms-ventas `VentaService.crearPedido` / `confirmarPago` (Feign a catálogo e inventario)	`Ventas.test.jsx` · `VentaServiceTest`, `VentaRequestDTOTest`
RF-26 – RF-28	`pages/Contacto.jsx`, `Nosotros.jsx`, `Consejos.jsx`, `Tiendas.jsx`	`validaciones.test.js`, `Nosotros.test.jsx`
RF-29, RF-43	`pages/admin/Dashboard.jsx`, `Ordenes.jsx`, `utils/ventas.js` · ms-ventas `GET /ventas`, `/ventas/{id}`	`PanelAdmin.test.jsx`, `ventas.test.js` · `VentaServiceTest` (detalle por rol)
RF-30 – RF-33, RF-36, RF-37	`pages/admin/Inventario.jsx`, `utils/inventario.js` · ms-catalogoProductos y ms-inventario	`Inventario.test.jsx`, `inventario.test.js` · `ProductoServiceImplTest`, `InventarioServiceTest`, `DataInitializerTest`
RF-34, RF-35	`pages/admin/ListaUsuarios.jsx`, `NuevoUsuario.jsx` · ms-auth `UsuarioService`	`PanelAdmin.test.jsx`, `NuevoUsuario.test.jsx` · `UsuarioServiceTest`, `UsuarioSeguridadTest`
RF-39	`pages/vendedor/*`, `LayoutPublico.jsx` · reglas de rol en los 3 `SecurityConfig` y `@PreAuthorize`	`PanelVendedor.test.jsx` · `InventarioSeguridadTest`, `ProductoSeguridadTest`, `VentaSeguridadTest`
RF-40, RF-42	`pages/MisCompras.jsx` · `VentaService.verificarAcceso`	`MisCompras.test.jsx` · `VentaServiceTest` (venta ajena → 403)
RF-41	`App.jsx` (`RutaProtegida`) · `@PreAuthorize` ADMIN	`RutaProtegida.test.jsx`, `UsuarioSeguridadTest`
RF-44	`pages/admin/Clientes.jsx`, `Reportes.jsx`, `utils/reportes.js`	`reportes.test.js`, `ClientesReportes.test.jsx`
RF-45	Controladores V1 y V2, `SwaggerConfig` (grupos v1/v2)	Prueba manual en Swagger (Anexo 2)

10. Criterios de aceptación (resumen)

1. Los 4 microservicios y Eureka se levantan (`levantar-servicios.ps1`) y aparecen **UP** en Eureka. Cada uno publica su Swagger con los grupos **v1** y **v2**.
2. Un **cliente** puede registrarse, iniciar sesión, comprar (la venta queda PAGADA y el **stock baja**) y ver la orden en "Mis compras".
3. Un **vendedor** solo ve lista y detalle de productos y de órdenes. Cualquier escritura en la API devuelve **403**.
4. El **administrador** puede crear, editar y eliminar productos, ajustar stock, crear usuarios con rol y ver órdenes, clientes y reportes con datos reales.
5. Sin token, las operaciones protegidas responden **401**; un cliente no puede ver ni pagar ventas ajenas (**403**); un registro duplicado responde **409**.
6. Las pruebas automatizadas del frontend y del backend pasan y generan **reporte de cobertura**.
7. `npm run build` y `npm run lint` terminan **sin errores**, y ningún secreto queda en el repositorio.

---

11. Glosario

- **Carrito:** lista temporal de productos que un cliente desea comprar.
- **Orden / Venta:** registro de una compra en ms-ventas, con número (`ORD-n`), ítems, despacho, total y estado (PENDIENTE_PAGO o PAGADA).
- **Stock crítico:** nivel de existencias (por defecto 5) bajo el cual el inventario marca alerta de reposición.
- **Tarifa residencial / comercial:** precio aplicado según el tipo de cliente.
- **Borrado lógico:** el producto se marca como inactivo y deja de verse en la tienda, sin eliminarse de la base.
- **Token JWT:** credencial firmada que el frontend envía en cada petición para identificar al usuario y su rol.
- **Semilla:** datos iniciales que cada microservicio carga si su base está vacía (usuarios de prueba y 14 productos).
- **RBAC:** control de acceso basado en roles.
- **Cobertura:** porcentaje del código ejecutado por las pruebas.

Fin del documento.